

DS GA 1011 Assignment 3: Tokenizer & Inference with LLMs

Fall 2026

Where to find the code All the assignment code as well as this writeup are available on GitHub at:

<https://github.com/gregdurrett/nyu-dsga1011-a3>

Please `git clone` the repository. If there are any updates, we will notify you so you can `git pull`.

Statement on AI tools Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about language models, but using it directly to solve the problem is prohibited. See the syllabus for more details.

Submission You will submit five files to Gradescope: **pdf** file containing answers to written questions (part 1), **bpe.py** file (part 1), **test_tokenizer.py** file (part 1), **submission.py** (part 2), and **prompting_exercises.ipynb** (part 2).

For Part II, you will need to request access for downloading model weights from Hugging Face, so we highly recommend starting early. Granting model weight access (esp. Llama-2-7B) can take a few days. First go through the file **README.md** and use the **requirements.txt** file to set up the environment required for the class.

1 Part 1. BPE Tokenizer (35 points)

In this portion, you will implement a Byte-pair encoding (BPE) tokenizer is a technique of learning a useful tokenizer from a text corpus. BPE first initializes the set of tokens as all characters in some alphabet, and iteratively merges the most frequent pair of adjacent tokens (bigram) as a new token. Each iteration adds a new token to the vocabulary while retaining the existing tokens.

Problem (bpe_implementation): Q1. Implement the BPE algorithm (23 points)

You should implement BPE tokenizer by filling in `ASCIIBPETokenizer.merge()` as well as `encode()` and `decode()` in `part1/src/bpe.py`. Your implementation should pass all of the test cases in `part1/tests/test_tokenizer.py`. Autograder might run additional test cases to evaluate tokenizer validity.

We may expect a tokenizer to satisfy the properties below:

- Injective: for all $s \neq \hat{s}$, $\text{tokenize}(s) \neq \text{tokenize}(\hat{s})$

- Invertible: for any string s , $s = \text{detokenize}(\text{tokenize}(s))$
- Preserves concatenation: for two strings $s, \hat{s}$, $\text{tokenize}(s + \hat{s}) = \text{tokenize}(s) + \text{tokenize}(\hat{s})$.

However, all three properties are violated by the tokenizers of multiple popular language models. Answer the following questions.

Problem (tokenizer_properties): Q2. Properties of tokenizers (12 points)

- (4 points, code) Provide counterexamples showing how a popular, publicly-available tokenizer fails above properties. Write code in `part1/tests/test_tokenizer.py` so that the assertions in the following functions all pass:
 - `test_not_injective()`
 - `test_not_invertible()`
 - `test_not_preserving_concat()`
- (4 points, written) In your report, write down no more than two sentences about each test, describing the tokenizer you chose and the violation you identified.
- (4 points, written) Comment on why it is generally impossible to build non-trivial tokenizers that preserve concatenation.

2 Part 2. Prompt Engineering for Addition (65 points)

The goal of this coding problem is the following:

1. Give you hands-on experience with prompt engineering.
2. Understand the challenges involved in prompt engineering.

Specifically, we will use models from Hugging Face to teach the Llama-2-7B and Qwen3-8B model to add two positive 7-digit integers. You can use HPC You could also use other LLM inference services (e.g., lightning.ai (which provides some free credits) for this part).

Problem (zero_shot_addition): Q3. Zero-shot Addition (10 points)

We provided you with a notebook `prompting_exercises.ipynb`. In the first section, there are two examples of zero-shot addition: ten instances of two 1-digit addition and ten instances of two 7-digit additions.

- (2 points, written) Run the two examples. In your opinion, what are some factors that cause language model performance to deteriorate from 1 digit to 7 digits?
- (3 points, written) Play around with the config parameters:
 - What does each parameter represent? The parameters to be probed are: Temperature, Max Tokens, Top P, Top K, and Repetition Penalty.

- How does increasing each parameter change the generation?
- c. (2 points, written) Do 7-digit addition with Qwen3-8B model.
- How does the performance change?
 - What are some factors that cause this change?
- d. (3 points, written) Previously we gave our language model the prior that the sum of two 7-digit numbers must have a maximum of 8 digits (by setting `max_token=8`). What if we remove this prior by increasing the `max_token` to 20?
- Does the model still perform well?
 - What are some reasons why?

Problem (in_context_learning): Q4. In Context Learning (25 points)

In this part, we will try to improve the performance of 7-digit addition via in-context learning. We will only use LLaMa-2-7B from here on.

- a. (3 points, written) Using the baseline prompt (“Question: What is 3+7? Answer: 10
Question: What is a+b? Answer:”), check 7-digit addition for 10 pairs again.
- Compared to zero-shot 7-digit additions with maximum 8 tokens, how does the performance change when we use the baseline in-context learning prompt?
 - What are some factors that cause this change?
- b. (7 points, written) Now we will remove the prior on output length and re-evaluate the performance of our baseline one-shot learning prompt. We need to modify our post processing function to extract the answer from the output sequence.
- Describe an approach to modify the post processing function.
 - Compared to 2a, How does the performance change when we relax the output length constraint?
 - What are some factors that cause this change?
- c. (9 points, written) Let’s change our one-shot learning example to something more “in-distribution”. Previously we were using 1-digit addition as an example. Let’s change it to 7-digit addition ($1234567+1234567=2469134$).
- Evaluate the performance with `max tokens = 8`. Report the res, acc, mae, prompt length.
 - Evaluate the performance with `max tokens = 50`. Report the res, acc, mae, prompt length.
 - How does the performance change from 1-digit example to 7-digit example?

- d. (6 points, written) Let's look at a specific example with large absolute error.
- Run the cell at least 5 times. Does the error change with each time? Why?
 - Can you think of a prompt to reduce the error?
 - Why do you think it would work?
 - Does it work in practice? Why or why not?

Problem (promptathon): Q5. Prompt-a-thon! (30 points)

In this part, you will teach Llama-2-7b model to add two 7-digit numbers reliably! Note: You can utilize a `.txt` file to test a specific prompt, but make sure the final prompt is in `submission.py` to pass all test cases for full credit.

We will provide you with a `run_tests.py` file to test your submission. It will evaluate your prompt on 30 instances of 7-digit integer addition. Some instances are manually designed, and others are randomly generated using a random seed that is different from `prompting_exercises.ipynb`.

We will run `submission.py` on our end with an autograder to verify your results. You must pass all test cases for full credit. Please keep `max_tokens` at least 50, so that you're not giving the model a prior on the output length. **Do not forget to include your `hf_token` and your `netid` in `submission.py`; failure to do so will not allow your submission to be autograded.**

Also, do not try to hack the pre-processing and post-processing functions by including arithmetic operations. **The autograder can detect the use of arithmetic operations in these functions.**

Here's a list of functions you can modify:

- `your_hf_token`
- `your_netid`
- `your_prompt`
- `your_config`
- `your_pre_processing` (but no arithmetic operation. e.g., `f"a+b={a+b}"` is not allowed.)
- `your_post_processing` (but no arithmetic operation. e.g., `f"a+b={a+b}"` is not allowed.)

- a. (coding, 30 points) Use prompt to improve test performance. For full credit, either

- get average accuracy greater than 0.2, or
- get average mean absolute error less than $1e5$.

For testing, we recommend using `python run_tests.py`.

This homework was modified from previous offering of this course (offered by He He) as well as an assignment from Daphne Ippolito's course.